CSA0920 - PROGRAMMING WITH JAVA

ASSIGNMENT REPORT

https://github.com/192521143simats-sketch/CSA09-192521143-Assignment2

Submitted by

R SARVESH (192521143)

Under the Supervision of

VIJAYARAGAVAN



SIMATS ENGINEERING



SIMATS

Saveetha Institute of Medical And Technical Sciences

(Declared as Deemed to be University under Section 3 of UGC Act 1956)

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

NOVEBER 2026

Q1

Repeated + becomes increasingly expensive as the number of fields grows because String is immutable. Every loop iteration creates a new accumulated value and copies its existing prefix. StringBuilder and StringBuffer use expandable buffers, so their copying work grows approximately with the final text length. StringBuffer synchronizes its methods. Its local-use overhead may be reduced by JVM lock elimination. Short tickets can show negligible differences because compilation and measurement overhead are significant.

The contrasting workloads are one 8-field ticket and 10,000 tickets with 50 fields each. The additional workload is 100 long tickets with 200 fields each. Each field occupies 18 characters including its newline. The one-ticket case is batched 2,000 times for timing resolution. Each approach has three warmup batches and seven measured trials. Times below are mean +/- sample standard deviation per ticket.

Tickets	Fields	String ns/ticket	StringBuilder ns/ticket	StringBuffer ns/ticket
1	8	656.62 +/- 403.18	926.40 +/- 1142.73	1077.74 +/- 1062.22
10000	50	7296.82 +/- 953.52	954.07 +/- 128.79	2261.16 +/- 96.55
100	200	94621.14 +/- 13817.85	6379.86 +/- 4011.50	9705.14 +/- 1754.45

At 50 fields, StringBuilder is 7.65 times faster than repeated +. At 200 fields, the ratio is 14.83. This supports the prediction for larger workloads. The short workload has substantial relative variation and does not establish a universal speed order. Every approach produces identical text.

```
String text = "";
for (String field : fields) text = text + field + "\n";
StringBuilder builder = new StringBuilder();
for (String field : fields) builder.append(field).append('\n');
StringBuffer buffer = new StringBuffer();
for (String field : fields) buffer.append(field).append('\n');
```

Logical allocation and character-copy estimates per ticket are as follows.

Fields	Final characters	String results	Intermediate results	+ characters copied	Builder appended characters
8	144	8	7	648	144
50	900	50	49	22950	900
200	3600	200	199	361800	3600

String-result counts describe the concatenation algorithm, rather than exact physical heap allocations. JVM optimization can change allocation behaviour. Each builder uses a buffer object, expanding arrays and a final String. Its final toString copy and geometric buffer growth add linear work beyond the appended-character count.

For n fixed-length fields of length L, repeated + copies $L(1 + 2 + ... + n) = Ln(n + 1)/2$ characters, giving $O(n^2)$ time and $O(n^2)$ cumulative intermediate character storage allocated. Its live working space can remain O(n) when obsolete strings are reclaimed. A builder requires amortized O(n) time and O(n) space, including the final String. With variable-length fields and total output length C, builder time is O(C), while repeated + costs the sum of all prefix lengths, bounded by O(nC).

Mean used-heap readings for the 100-ticket, 200-field workload are in MiB.

Approach	Before MiB	After MiB
String	139.12	139.54
StringBuilder	113.38	114.60
StringBuffer	125.88	127.10

Heap readings include garbage collection and allocation effects. An after value below a before value indicates collection and cannot establish negative allocation. These samples do not measure retained memory or total allocated bytes. The deterministic prefix-copy counts explain allocation pressure more reliably.

Use a thread-confined StringBuilder for each ticket. For a shared in-memory booking log, StringBuffer can append each complete record with one synchronized append call. A record composed through several append calls requires an external synchronized block to prevent interleaving. Four concurrent writers appending 1,000 complete records each produce 4,000 intact lines.

The alternatives are as follows.

```
String ticket = String.join("\n", fields) + "\n";
StringJoiner joiner = new StringJoiner("\n", "", "\n");
fields.forEach(joiner::add);
String ticket2 = joiner.toString();
String ticket3 = String.format("%s%n", String.join("\n", fields));
```

String.join and StringJoiner take O(C + n) time and O(C + n) storage for n fields and C output characters. String.format adds format parsing and conversion overhead. For this fixed format it remains linear in the output length. %n uses the platform newline. join and StringJoiner produce the same ticket text as the builder. Formatting is useful when values need width, precision or locale conversion.

Q2

ArrayList is appropriate for append-heavy playlists, frequent jumps to the k-th song and full traversal. LinkedList is appropriate for frequent endpoint changes or insertion through an already positioned iterator. Indexed middle editing still requires traversal in LinkedList.

Predicted costs are amortized constant-time appends for both lists, linear array shifting for front and middle edits, constant-time ArrayList indexed reads, linear LinkedList indexed traversal and linear full iteration for both.

The workloads use 100, 10,000 and 100,000 Song records. Each trial contains 500 appends, 500 insertion/removal pairs at the front, 500 pairs in the middle, 500 seeded get(k) queries and a complete enhanced-for traversal. An edit pair restores the original playlist. Setup and full order checks are outside the edit timer. Three warmups precede seven measured trials. The table gives mean +/- sample standard deviation in ns per append, edit pair, indexed read or iterated element.

Songs	Operation	ArrayList ns	LinkedList ns
100	append	48.71 +/- 7.53	63.54 +/- 31.64
100	front-edit-pair	267.03 +/- 91.92	239.00 +/- 74.18
100	middle-edit-pair	204.03 +/- 8.76	571.40 +/- 268.01
100	get-k	73.57 +/- 8.39	189.31 +/- 143.45
100	iterate	91.29 +/- 4.03	130.00 +/- 44.63

Songs	Operation	ArrayList ns	LinkedList ns
10000	append	54.69 +/- 7.60	42.49 +/- 14.95
10000	front-edit-pair	519.77 +/- 17.08	91.43 +/- 14.65
10000	middle-edit-pair	335.66 +/- 38.54	12453.09 +/- 136.06
10000	get-k	71.77 +/- 2.78	3273.69 +/- 248.47
10000	iterate	39.36 +/- 26.15	13.27 +/- 3.75
100000	append	280.86 +/- 10.39	35.40 +/- 6.38
100000	front-edit-pair	10884.66 +/- 174.78	15.69 +/- 0.72
100000	middle-edit-pair	5298.26 +/- 168.17	129541.23 +/- 2803.96
100000	get-k	61.09 +/- 6.76	32414.51 +/- 169.23
100000	iterate	14.26 +/- 6.17	10.72 +/- 2.04

At 100,000 songs, LinkedList indexed reads cost approximately 530.64 times the ArrayList mean. ArrayList front pairs require 100,000,000 reference shifts across 500 pairs. Middle pairs require approximately 50,000,000 shifts. The corresponding LinkedList middle searches need approximately 50,000,000 node steps.

A practical boundary is a 10-microsecond mean budget per edit pair for collection work. An operation mix that exceeds that budget at its actual playlist size should trigger a redesign. The table identifies which measured sizes and operations cross it. This budget excludes playback, UI and network work. No universal playlist-size cutoff follows from one machine's timings.

For Up Next, ArrayDeque supports additions at both ends and front removal without allocating linked nodes per addition. Each measured cycle makes two additions and two front removals.

Songs	ArrayDeque cycle ns	LinkedList cycle ns
100	217.46 +/- 30.76	226.34 +/- 9.45
10000	345.80 +/- 44.45	211.69 +/- 73.98
100000	625.20 +/- 35.40	199.09 +/- 15.90

ArrayDeque is the preferred endpoint-only queue because of its compact storage and amortized constant endpoint operations. A deque constructed at full occupancy can resize during the first cycle, and that cost is included in these batches. Preallocating spare capacity changes the steady-state comparison. Actual timings and their deviations therefore qualify any speed claim.

Operation	**ArrayList**	**LinkedList**	**ArrayDeque**
Append	O(1) amortized, O(n) resize	O(1)	O(1) amortized, O(n) resize
Front insert/remove	O(n)	O(1)	O(1) amortized insert, O(1) remove
Indexed middle insert	O(n)	O(n) search, O(1) link change	No indexed API, O(n) manual rearrangement
Get k-th	O(1)	O(min(k, n-k))	No indexed API, O(n) traversal
Full iteration	O(n)	O(n)	O(n)

All structures need O(n) storage. ArrayList and ArrayDeque use reference slots and spare capacity. LinkedList allocates a node with an element reference and two link references per song. With common compressed-reference HotSpot layouts, an array reference slot is about 4 bytes and a linked node about 24 bytes. Exact sizes depend on the JVM, and Song objects are additional in all designs.

A manual array solution keeps Song[] data and a size counter. Append stores data[size++] and doubles capacity when full. Insertion shifts the suffix right with System.arraycopy(data, k, data, k + 1, size - k). Removal shifts left and clears the last slot. Bounds-checked access returns data[k]. Its complexities match ArrayList, including amortized O(1) append, O(n) shifting and O(n) capacity space. A fixed-capacity array must reject insertion when full. Empty get(0) is invalid and must raise an index exception.

Q3

Equal students must have equal hash codes. Registration number defines attendance identity, so both equals and hashCode must depend on the same immutable regNo value. The name does not determine uniqueness.

Scans or operation	**Predicted result and reason**	**Observed result in seven repetitions**
Reuse the same Arun object twice, plus Divya	Size 2, because the same reference retains its identity hash	Size 2
Use distinct Arun instances with the same registration and deliberately different identity hashes, plus Divya	Size 3, because equal objects have inconsistent hashes	Size 3
Insert mutable registration A, change it to B, then query and remove	Size remains 1, but the new hash cannot find the old entry	Size 1, contains false, remove false
Reinsert the mutated object	Size 2, because the old insertion hash still owns its original entry	Size 2
Use immutable regNo with a consistent hashCode	Size 2 for the three scans	Size 2

Identity hashes can collide. Deliberately choosing duplicate objects with different hashes makes the failing test reliable. Reusing the same reference explains why the broken class can appear to work correctly.

HashSet uses a HashMap internally. It spreads hashCode, selects a bucket, and compares eligible keys using reference identity or equals. It does not compare a new key with every key in the entire set. Equal registrations with different hashes can therefore occupy separate entries. Unequal objects may share a hash, provided equals can distinguish them.

A key's insertion hash remains stored in its entry. Changing an equality field does not move the entry. The A-to-B mutation selects a different hash location, stranding the original entry. Remove a key before changing its equality fields and then reinsert it, or use immutable equality fields.

The corrected class is as follows.

```
class Student {
   private final String regNo;
   private final String name;
   Student(String regNo, String name) {
      this.regNo = regNo;
      this.name = name;
   }
   @Override public boolean equals(Object o) {
      if (this == o) return true;
      if (!(o instanceof Student)) return false;
      return regNo.equals(((Student) o).regNo);
   }
   @Override public int hashCode() {
      return regNo.hashCode();
   }
   @Override public String toString() {
      return regNo + " " + name;
   }
}
```

The supplied program's output before correction is Unique students: 3. After correction it is Unique students: 2, matching the expected output. The corrected identity is reflexive, symmetric and transitive, rejects null and other types, gives equal hashes for equal registrations and supports lookup with a new equal object. A changed name with the same registration remains equal. This class assumes a non-null registration.

A TreeSet ordered by registration de-duplicates when its comparator returns zero for equal registration numbers. A HashMap<String, Student> keyed by registration can use putIfAbsent to retain the first scan. Using accessors for the private registration field, the alternatives are as follows.

ordered = TreeSet(comparator on student registration)
for each scan: ordered.add(scan)
byRegistration = HashMap()
for each scan: byRegistration.putIfAbsent(scan.registration, scan)

Structure	Time per scan	Space for n unique students
Corrected HashSet	O(1) average, O(n) worst-case contractual bound	O(n) table and entries
TreeSet by regNo	O(log n) comparisons	O(n) tree nodes
HashMap keyed by regNo	O(1) average, O(n) worst-case contractual bound	O(n) table, entries and key references

Both alternatives retain two registrations. A name-only comparator would incorrectly merge different students sharing a name. String hashing and comparison also depend on registration length, which is fixed and short here.

Q4

Use an insertion-ordered LinkedHashMap<String, Medicine> for fast stock lookup and the addition-order report. Store each medicine's original addition sequence so another map can reconstruct that order. Preserve that sequence when changing stock.

Map	Put	Get	Iterate	Native order	Extra report work
HashMap	O(1) average	O(1) average	O(n + capacity)	Unspecified	Sort either order in O(n log n)
TreeMap	O(log n)	O(log n)	O(n)	Code order	Sort by sequence for addition order
LinkedHashMap	O(1) average	O(1) average	O(n)	Addition order	Sort by code for code order

The acceptable mean collection lookup time is 10 microseconds. Sizes are 100, 10,000 and 100,000 medicines from the shuffled input.csv. Each lookup trial contains 100,000 successful, seeded requests using equal but separately allocated code strings. Three warmups precede seven trials. Report timings include sorting where required and rendering every row, while file/printer I/O is excluded. Each

column below gives mean +/- sample standard deviation in milliseconds. Lookup values are whole 100,000-request batches, and report values are whole reports.

Medicines	Map	100000 lookups ms	Code report ms	Addition report ms
100	HashMap	1.59 +/- 0.40	0.25 +/- 0.24	0.23 +/- 0.38
100	TreeMap	6.00 +/- 0.76	0.04 +/- 0.01	0.09 +/- 0.02
100	LinkedHashMap	2.09 +/- 0.26	0.06 +/- 0.00	0.07 +/- 0.11
10000	HashMap	8.34 +/- 4.38	7.97 +/- 1.99	7.17 +/- 3.36
10000	TreeMap	17.34 +/- 0.96	0.93 +/- 0.10	2.87 +/- 0.86
10000	LinkedHashMap	2.62 +/- 0.62	2.69 +/- 0.34	0.29 +/- 0.08
100000	HashMap	11.50 +/- 2.08	26.67 +/- 12.73	32.05 +/- 13.11
100000	TreeMap	52.54 +/- 1.46	15.97 +/- 3.42	30.11 +/- 1.36
100000	LinkedHashMap	11.39 +/- 3.21	45.27 +/- 18.31	4.17 +/- 0.81

At 100,000 medicines, the mean lookup costs are 114.98 ns for HashMap, 525.35 ns for TreeMap and 113.91 ns for LinkedHashMap. All meet the criterion. A mean is not a worst-case or percentile guarantee.

LinkedHashMap retains addition order directly and needs sorting only for the evening code report. TreeMap makes the code report cheaper but needs extra ordering work for addition order. HashMap needs ordering work for both. All nine map/size combinations match both required report orders. Missing keys return null. Access-order LinkedHashMap must not be used because get would change iteration order.

The remaining report cost comes from comparisons, traversal, string formatting, allocation and garbage collection. Real request handling, persistence, synchronization and I/O add costs beyond these measurements. All maps use O(n) storage. LinkedHashMap adds order links, TreeMap adds tree links and balancing data, and hash tables reserve empty buckets. Report sorting uses O(n) references, while its text needs O(C) space for C output characters. Hash-map operations have an O(n) worst-case contractual bound.

Hashtable is a legacy synchronized map. Per-method locking can add contention and does not make a multi-step inventory update atomic. ConcurrentHashMap or explicit synchronization with consistent reporting snapshots is more suitable when concurrent access is required.

For a sorted-array alternative, sort Medicine[] by code and call Arrays.binarySearch with the same code comparator. A negative result identifies insertion position -result - 1. Grow the array if necessary and shift its suffix right before inserting. Existing codes replace stock while retaining their addition sequence. Code-order reporting traverses the array directly.

Medicines	**Sorted-array lookup ns mean +/- SD**
100	66.59 +/- 11.48
10000	184.23 +/- 16.27
100000	351.64 +/- 7.15

Sorted-array operation	**Complexity**
Lookup	O(log n) time, O(1) search space
Insert	O(log n) search plus O(n) shift
Initial build from unsorted data	O(n log n)
Code-order report	O(n + C) including text rendering
Addition-order report	O(n log n + C) unless a second order index is retained
Storage	O(n) reference slots plus medicine objects

The array has less node overhead but expensive insertion. TreeMap also offers O(log n) lookup with O(log n) insertion. HashMap offers O(1) average lookup and insertion, with an O(n log n) code-order sort when needed.

Q5

Use a separate Comparator<Team> because ranking policy should be reusable and replaceable. Comparable<Team> is suitable when one permanent natural order belongs to Team. A comparator allows the same Leaderboard<T> design to rank teams, players, colleges and quiz teams.

The ranking key is descending points, descending goal difference and ascending name. Each component applies only when every earlier component ties.

```
Comparator<Team> byRank = Comparator
   .comparingInt((Team t) -> t.points).reversed()
   .thenComparing(Comparator
      .comparingInt((Team t) -> t.goalDiff).reversed())
   .thenComparing(t -> t.name);
```

The generic ranking component is as follows.

```
class Leaderboard<T> {
   private final List<T> items = new ArrayList<>();
   private final Comparator<? super T> order;
   Leaderboard(Comparator<? super T> order) {
      this.order = order;
   }
   void add(T item) { items.add(item); }
   List<T> ranking() {
      List<T> sorted = new ArrayList<>(items);
      sorted.sort(order);
      return sorted;
   }
}
```

Comparator<? super T> accepts a comparator for T or its supertype. No Comparable bound is needed with an injected comparator. A natural-order method instead requires <T extends Comparable<? super T>>. A producer wildcard List<? extends T> lets an addAll helper read subclasses as T without allowing arbitrary T values to be inserted into that input list.

```
static <T> void addAll(Leaderboard<T> board, List<? extends T> input) {
   for (T item : input) board.add(item);
}
```

The supplied output is exactly [Eagles(9,6), Bisons(9,4), Panthers(9,4), Cobras(6,1)]. Bisons precedes Panthers because both have 9 points and goal difference 4, so alphabetical name order resolves the tie.
The extended tie dataset ranks Eagles, Falcons, Bisons, Bisons, Panthers, Tigers, Aardvarks, Cobras. It contains ties in points, ties in goal difference and two distinct entries with identical ranking fields. Those identical ranking keys compare as zero, while both entries remain in the List. The supplied Team class retains identity-based Java equals, so ranking equivalence should not automatically be used as domain identity in a TreeSet.
The extreme-score dataset ranks Maximum(2147483647,0), Zero(0,0), Minimum(-2147483648,0). The faulty comparison b.points - a.points overflows for extreme scores and can reverse an ordering decision. Omitting the name comparison incorrectly leaves Bisons and Panthers tied. The correct chain satisfies pair and triple checks on both datasets, while the faulty subtraction violates comparison laws.
Correctness follows from lexicographic transitivity. Integer comparison and String comparison are transitive, and reversing a numeric order preserves transitivity. For A before B and B before C, the earliest component where either comparison differs orders A before C. Later components cannot reverse it. Equal keys agree in all components and therefore compare equally against every third key. Reversing each numeric comparator separately preserves ascending names. Reversing the whole completed chain would also reverse names.
Comparable can implement the same ordering through compareTo returning byRank.compare(this, other). A bounded natural-ranking helper copies the list and calls Collections.sort. The comparator version copies and calls List.sort. A second alternative uses three stable sorts on a copy, first name ascending, then goal difference descending, then points descending. Stability preserves the priorities established by earlier passes within later ties.

Approach	**Sort work**	**Peak extra space**	**Reuse**
Comparator chain	One O(n log n) sort plus O(n) copy	O(n) copy and sorting workspace	Same generic board, new comparator per type
Comparable	One O(n log n) sort plus O(n) copy	O(n)	Natural-order policy belongs to each type
Three stable sorts	Three O(n log n) sorts, still O(n log n) overall	O(n), because passes are sequential	More passes and less concise policy

With three warmups and seven measured trials, full ranking times are as follows in milliseconds, mean +/- sample standard deviation.

Items	Comparator ms	Comparable ms	Three stable sorts ms
100	0.06 +/- 0.01	0.20 +/- 0.21	0.72 +/- 0.83
1000	0.51 +/- 0.14	0.41 +/- 0.05	0.44 +/- 0.07
10000	5.01 +/- 1.39	5.69 +/- 1.40	3.86 +/- 1.50

All alternatives give the same ranking. Clearing a returned list leaves the board unchanged, and an empty board returns an empty list. Timing variation prevents a universal constant-factor claim between implementations. Comparator chains provide the most direct reusable policy. Name-comparison cost also depends on name length.

Q6

A PriorityQueue ordered by severity ascending and arrival counter ascending enforces emergency priority with FIFO ties. A plain LinkedList or ArrayDeque used as FIFO cannot move a newly arrived critical patient ahead of an earlier minor patient.

```
private final PriorityQueue<Patient> queue = new PriorityQueue<>(
    Comparator.comparingInt((Patient p) -> p.severity)
        .thenComparingLong(p -> p.arrival));
```

Each admit assigns counter++ once. Equal severities therefore have distinct arrival keys, and the smaller counter is served first. poll removes the next patient and returns null when empty. The supplied output is exactly Meena(S1) Kumar(S1) Lata(S2) Ravi(S3) John(S3) Asha(S5).

For arrivals Ravi S3, Meena S1, John S3, Asha S5, Kumar S1, Lata S2, Priya S4 and Omar S2, the hand-derived and observed treatment order is Meena, Kumar, Lata, Omar, Ravi, John, Priya, Asha. The arrival counter resolves the three severity ties correctly.

Display the waiting list by copying the PriorityQueue and repeatedly polling that copy. The original retains all eight patients. Direct iteration exposes heap storage, where only parent-child ordering is guaranteed. A separate eight-entry witness iterates severities 1, 4, 2, 5, 4, 3, 2, 5, demonstrating that direct printing need not show treatment order.

Predicted heap work is O(log n) for admission and removal, with occasional O(n) capacity expansion. The growth workloads contain 100, 10,000 and 50,000 pre-created patients. Capacity reservation isolates heap work in admission timing. Seven trials follow three warmups. Admission is averaged over a growing queue, and removal over a full drain. Times are mean +/- sample standard deviation in ns per patient.

Patients	Admit ns	Serve ns	Add comparisons	Drain comparisons	Single critical add comparisons
100	229.14 +/- 77.93	634.00 +/- 257.15	164	837	6
10000	109.33 +/- 59.15	226.49 +/- 58.39	18153	216496	13
50000	29.25 +/- 10.35	210.86 +/- 12.71	90746	1314833	15

Random admission uses about 1.6 to 1.8 comparisons per patient and does not demonstrate worst-case insertion growth. One critical arrival into a queue of minor patients needs 6, 13 and 15 comparisons, matching floor(log2(n + 1)). Drain comparisons per patient grow from 8.37 to 21.65 to 26.30. These counts support logarithmic removal and worst-case admission more directly than noisy timing means.

A severity-5 patient can starve whenever a more severe patient remains available at every service decision. A fair aging rule sets effectiveSeverity = max(1, originalSeverity - floor(waitMinutes / 30)), breaking ties by original arrival. After 120 minutes, an original severity-5 patient reaches effective severity 1 and precedes a newly arrived severity-1 patient. With finite earlier backlog and continuing service, later arrivals can no longer repeatedly overtake that promoted patient.

Compute immutable effective keys at a fixed snapshot time and rebuild the heap when refreshing priorities. A comparator that silently changes with the clock would invalidate the heap's ordering. Repeated-add rebuilding costs O(n log n) time and O(n) temporary space. Refreshing periodically trades promotion delay for lower overhead. This rule changes strict original-severity priority, so its threshold is an explicit policy choice.

At 50,000 queued patients, the measured mean admission plus removal cost implies about 1.20 ms of collection work for 5,000 admit/serve pairs. The queue algorithm can support that daily count under the measured conditions. This estimate excludes patient allocation, request processing, persistence, I/O and aging rebuilds. A shared deployed PriorityQueue and its counter require synchronized or serialized access.

Design	Admit	Serve next	Space
PriorityQueue	O(log n) heap work, occasional O(n) growth	O(log n)	O(n) references and patients
Sorted LinkedList	O(n) search, O(1) positioned insertion	O(1) removeFirst	O(n) linked nodes and patients
Five FIFO queues	O(1) amortized addLast	At most five checks, O(1)	O(n + 5)

The sorted list advances a ListIterator to the correct comparator position, then inserts there. The five-queue alternative appends to queues[severity - 1] and removes from the first non-empty queue when scanning severity 1 through 5. Both give the same eight-patient treatment order. Five queues suit fixed severity levels but require explicit movement or rebuilding for aging. Severity outside 1 through 5 is invalid, and an empty poll returns null.

Q7

A. The bottleneck is repeated sorting in Q5's ranking method. Every request copies and sorts the complete items list, even when no team has changed. The 10,000-team comparator ranking averages 5.010 ms. Repeating the same request repeats its comparisons and temporary sorting allocations. Replacing the collection alone would not remove this work. An exact time split between copying and sorting would require profiling.

B. The objective is at least 90% fewer full sorts and a twofold lower mean request time for 30 unchanged requests at n >= 1,000, while preserving ranking correctness, alphabetical ties, independent mutable results and visibility of additions in the next ranking.

C. Retain the items list and comparator, and add a private sorted cache with a dirty flag. add marks the cache dirty. The next ranking rebuilds its sorted copy, while later requests copy the cached order until another addition occurs.

```
add(team)
    items.append(team)
    dirty = true

ranking()
    if dirty
        refreshed = copy(items)
        sort(refreshed, sameComparator)
        sorted = refreshed
        dirty = false
    return copy(sorted)
```

Immutable team records make the cache valid between updates. Every supported addition invalidates it. Future removals and score changes must also invalidate it. Externally mutable ranking keys would require explicit invalidation. Concurrent access needs a common lock or a separately designed publication mechanism.

The cache retains O(n) additional references and still creates an O(n) defensive result copy per query. A refresh adds another O(n) cache copy. Returning a shared unmodifiable list could avoid defensive copying but would change the original mutable-result contract.

D. For 30 unchanged requests, the predicted sort counts are 30 for the original and 1 for the cache, a 96.67% reduction. An update before requests 0, 10 and 20 gives three cached sorts, a 90% reduction. Updating before every request gives 30 sorts in both designs and no comparison-work saving. A twofold lower mean is predicted for unchanged data at n >= 1,000. Cache refresh overhead may make update-heavy workloads slower.

E. Both designs use identical seeded teams, updates, comparators and 30-request conditions at 100, 1,000 and 10,000 initial teams. Each trial starts with a cold cache. Three warmup batches precede seven trials. Timed work includes the required additions, rankings and traversal of every returned entry. Every result agrees with the original, client-side clearing cannot change the cache, and earlier snapshots retain their old contents.

Workload at 10000 teams	Original sorts	Cached sorts	Original comparisons	Cached comparisons
No updates	30	1	3456420	115214
Update every 10 requests	30	3	3458020	345802
Update every request	30	30	3459182	3459182

Interval 0 means no updates. Times are mean +/- sample standard deviation in ms per request, including the cold-cache first query within its batch. Ratio means original mean divided by cached mean.

Teams	Update interval	Original ms	Cached ms	Ratio
100	0	0.0391 +/- 0.0329	0.0091 +/- 0.0013	4.29
100	10	0.0243 +/- 0.0055	0.0037 +/- 0.0008	6.60
100	1	0.0178 +/- 0.0034	0.0168 +/- 0.0044	1.06
1000	0	0.2251 +/- 0.0372	0.0586 +/- 0.0407	3.84
1000	10	0.1454 +/- 0.0060	0.0173 +/- 0.0024	8.43
1000	1	0.1610 +/- 0.0206	0.1701 +/- 0.0364	0.95
10000	0	2.1966 +/- 0.0591	0.0907 +/- 0.0111	24.22
10000	10	2.2589 +/- 0.0623	0.2518 +/- 0.0312	8.97
10000	1	2.1736 +/- 0.0411	2.2385 +/- 0.0784	0.97

F. The unchanged-query sort objective is achieved, with 96.67% fewer sorts. Mean request time improves by factors of 3.84 at 1,000 teams and 24.22 at 10,000 teams, meeting the timing objective in this run. Updating every tenth request avoids 90% of sorts. Updating every request avoids none, so incidental timing differences in that condition cannot establish an algorithmic caching benefit.

For R requests, U dirty versions and a roughly fixed size n, original time is O(R n log n), while cached time is O(U n log n + R n). Unchanged data has U = 1. An update before every request has U = R,

leaving the same asymptotic cost and extra refresh-copy overhead. Both use O(n) peak auxiliary space, but the cache retains an additional O(n)-reference array. Snapshots retained by callers add memory in both designs.

Used caching for multiple reads between updates. Prefered the original for small or one-off rankings, frequent updates followed by one read, or a tighter retained-memory budget. Fixed method order, JIT compilation, GC and scheduling can affect measured ratios, so deterministic comparison counts remain the clearest evidence of avoided work.